101.2 Arranque del sistema

Referencia al objetivo del LPI

LPIC-1 v5, Exam 101, Objective 101.2

Importancia

3

Áreas de conocimiento clave

- Proporcionar comandos comunes al gestor de arranque y opciones al kernel en el momento del arranque.
- Demostrar conocimiento de la secuencia de arranque desde BIOS/UEFI hasta la finalización del arranque.
- Comprensión de SysVinit y systemd.
- Conocimiento de Upstart.
- Comprobar los eventos de arranque en los archivos de registro.

Lista parcial de archivos, términos y utilidades

- `dmesg`
- `journalctl`
- BIOS
- UEFI
- bootloader
- kernel
- `initramfs`
- `init`
- SysVinit
- systemd

101.2 Lección 1

Certificación: LPIC-1
Versión: 5.0
Tema: 101 Arquitectura de Sistema
Objetivo: 101.2 Arranque de sistema
Lección: 1 de 1

Introducción

Para controlar la máquina, el componente principal del sistema operativo, el núcleo, debe cargarse mediante un programa llamado *bootloader*, que a su vez lo carga un firmware preinstalado como BIOS o UEFI. El gestor de arranque se puede personalizar para pasar parámetros al núcleo, como qué partición contiene el sistema de archivos raíz o en qué modo debe ejecutarse el sistema operativo. Una vez cargado, el núcleo continúa el proceso de arranque identificando y configurando el hardware. Por último, el núcleo del sistema operativo llama a la utilidad responsable de iniciar y administrar los servicios del sistema.

Note En algunas distribuciones de Linux, los comandos ejecutados en esta lección pueden requerir privilegios de root.

BIOS o UEFI

Los procedimientos ejecutados por las máquinas x86 para ejecutar el gestor de arranque son diferentes si usa BIOS o UEFI. El BIOS, abreviatura de *Basic Input/Output System*, es un programa almacenado en un chip de memoria no volátil conectado a la placa base, que se ejecuta cada vez que se enciende la computadora. Este tipo de programa se llama *firmware* y su ubicación de almacenamiento es independiente de los otros dispositivos de almacenamiento que pueda tener el sistema. El BIOS supone que los primeros 440 bytes en el primer dispositivo corresponden a almacenamiento, siguiendo el orden definido en la utilidad de configuración del BIOS, la primera etapa del cargador de arranque (también llamado *bootstrap*). Los primeros 512 bytes de un dispositivo de almacenamiento se denominan MBR (*Master Boot Record*), en dispositivos de almacenamiento que utilizan el esquema de partición estándar de DOS y, además de la primera etapa del gestor de arranque, contiene la tabla de particiones. Si el MBR no contiene los datos correctos, el sistema no podrá arrancar, a menos que se emplee un método alternativo.

En términos generales, los pasos previos a la operación para iniciar un sistema equipado con BIOS son:

1. El proceso POST (*power-on self-test*) se ejecuta para identificar fallas de dispositivos simples tan pronto como se enciende la máquina.
2. El BIOS activa los componentes básicos para cargar el sistema, como salida de video, teclado y medios de almacenamiento.
3. El BIOS carga la primera etapa del gestor de arranque desde el MBR (los primeros 440 bytes del primer dispositivo, como se define en la utilidad de configuración del BIOS).

4. La primera etapa del gestor de arranque llama a la segunda etapa del gestor de arranque, responsable de presentar las opciones de arranque y cargar el núcleo del sistema operativo.

El UEFI, abreviatura de *Unified Extensible Firmware Interface*, difiere del BIOS en algunos puntos claves. Como BIOS, el UEFI también es un firmware, pero puede identificar particiones y leer muchos sistemas de archivos que se encuentran en ellas. El UEFI no se basa en el MBR, teniendo en cuenta solo la configuración almacenada en su memoria no volátil (*NVRAM*) conectada a la placa base. Estas definiciones indican la ubicación de los programas compatibles con UEFI, llamados *EFI applications*, que se ejecutarán automáticamente o se llamarán desde un menú de arranque. Las aplicaciones EFI pueden ser gestores de arranque, selectores de sistema operativo, herramientas para el diagnóstico y reparación del sistema, etc. Deben estar en una partición de dispositivo de almacenamiento convencional y en un sistema de archivos compatible. Los sistemas de archivos compatibles estándar son FAT12, FAT16 y FAT32 para dispositivos de bloque e ISO-9660 para medios ópticos. Este enfoque permite la implementación de herramientas mucho más sofisticadas que las posibles con BIOS.

La partición que contiene las aplicaciones EFI se llama *EFI System Partition* o simplemente ESP. Esta partición no debe compartirse con otros sistemas de archivos del sistema, como el sistema de archivos raíz o los sistemas de archivos de datos del usuario. El directorio EFI en la partición ESP contiene las aplicaciones señaladas por las entradas guardadas en la NVRAM.

En términos generales, los pasos de arranque del sistema preoperativo en un sistema con UEFI son:

1. El proceso POST (*power-on self-test*) se ejecuta para identificar fallas de dispositivos simples tan pronto como se enciende la máquina.
2. El UEFI activa los componentes básicos para cargar el sistema, como salida de video, teclado y medios de almacenamiento.
3. El firmware de UEFI lee las definiciones almacenadas en NVRAM para ejecutar la aplicación EFI predefinida almacenada en el sistema de archivos de la partición ESP. Por lo general, la aplicación EFI predefinida es un gestor de arranque.
4. Si la aplicación EFI predefinida es un gestor de arranque, cargará el núcleo para iniciar el sistema operativo.

El estándar UEFI también admite una característica llamada *Secure Boot*, que solo permite la ejecución de aplicaciones EFI firmadas, es decir, aplicaciones EFI autorizadas por el fabricante del hardware. Esta característica aumenta la protección contra software malicioso, pero puede dificultar la instalación de sistemas operativos no cubiertos por la garantía del fabricante.

El Cargador de Arranque

El gestor de arranque más popular para Linux en la arquitectura x86 es GRUB (*Grand Unified Bootloader*). Tan pronto como lo llame el BIOS o el UEFI, GRUB muestra una lista de los sistemas operativos disponibles para arrancar. A veces, la lista no aparece

automáticamente, pero se puede invocar presionando `Shift` mientras el BIOS está llamando a GRUB. En los sistemas UEFI, la tecla `Esc` debería usarse en su lugar.

Desde el menú GRUB es posible elegir cuál de los núcleos instalados debe cargarse y pasarle los parámetros. La mayoría de los parámetros del núcleo siguen el patrón `option=value`. Algunos de los parámetros del núcleo de Linux más útiles son:

`acpi`

Habilita/deshabilita el soporte ACPI. `acpi=off` deshabilitará la compatibilidad con ACPI.

`init`

Establece un iniciador de sistema alternativo. Por ejemplo, `init=/bin/bash` establecerá shell Bash como iniciador. Esto significa que se iniciará una sesión de shell justo después del proceso de arranque del kernel.

`systemd.unit`

Establece a *systemd* para que se active. Por ejemplo, `systemd.unit=graphical.target`. Systemd también acepta los niveles de ejecución numéricos definidos para *SysV*. Para activar el nivel de ejecución 1, por ejemplo, solo es necesario incluir el número `1` o la letra `S` (abreviatura de "single") como parámetro del núcleo.

`mem`

Establece la cantidad de RAM disponible para el sistema. Este parámetro es útil cuando se utilizan máquinas virtuales, limitando la cantidad de RAM disponible para cada una de ellas. El uso de `mem=512M` limitará a 512 megabytes la cantidad de RAM disponible para un sistema virtual en particular.

`maxcpus`

Limita el número de procesadores (o núcleos de procesador) visibles para el sistema en máquinas simétricas multiprocesador. También es útil para máquinas virtuales. Un valor de `0` desactiva el soporte para máquinas multiprocesador y tiene el mismo efecto que el parámetro del núcleo `nosmp`. El parámetro `maxcpus=2` limitará el número de procesadores disponibles para el sistema operativo a dos.

`quiet`

Oculta la mayoría de los mensajes de arranque.

`vga`

Selecciona un modo de video. El parámetro `vga=ask` mostrará una lista de los modos disponibles para elegir.

`root`

Establece la partición raíz, distinta de la preconfigurada en el gestor de arranque. Por ejemplo, `root =/dev/sda3`.

`rootflags`

Opciones de montaje para el sistema de archivos raíz.

`ro`

Hace que el montaje inicial del sistema de archivos raíz sea de solo lectura.

`rw`

Permite escribir en el sistema de archivos raíz durante el montaje inicial.

Por lo general, no es necesario cambiar los parámetros del núcleo de Linux, pero puede ser útil para detectar y resolver problemas relacionados con el sistema operativo. Los parámetros del núcleo del sistema operativo deben agregarse al archivo `/etc/default/grub` en la línea `GRUB_CMDLINE_LINUX` para que sean persistentes durante los reinicios. Se debe generar un nuevo archivo de configuración para el gestor de arranque cada vez que `/etc/default/grub` cambie, lo cual se logra mediante el comando `grub-mkconfig -o /boot/grub/grub.cfg`. Una vez que el sistema operativo se está ejecutando, los parámetros del núcleo utilizados para cargar la sesión actual están disponibles en el archivo `/proc/cmdline`.

Note La configuración de GRUB se discutirá en próximas lecciones.

Inicialización del Sistema

Además del núcleo, el sistema operativo depende de otros componentes que proporcionan las características esperadas. Muchos de estos componentes se cargan durante el proceso de inicialización del sistema, que varía desde simples scripts de consola hasta programas de servicio más complejos. Las secuencias de comandos a menudo se utilizan para realizar tareas de corta duración que se ejecutarán y finalizarán durante el proceso de inicialización del sistema. Los servicios, también conocidos como demonios, pueden estar activos todo el tiempo, ya que pueden ser responsables de los aspectos intrínsecos del sistema operativo.

La diversidad de formas en que los scripts de inicio y los demonios con las características más diferentes se pueden integrar en una distribución de Linux es enorme, un hecho que históricamente obstaculizó el desarrollo de una solución única que cumpliera con las expectativas de los mantenedores y usuarios de todas las distribuciones de Linux. Sin embargo, cualquier herramienta que los encargados de la distribución hayan elegido para realizar esta función al menos podrá iniciar, detener y reiniciar los servicios del sistema. El sistema mismo suele realizar estas acciones después de una actualización de software, por ejemplo, pero el administrador del sistema casi siempre necesitará reiniciar manualmente el servicio después de realizar modificaciones en su archivo de configuración.

También es conveniente que un administrador del sistema pueda activar un conjunto particular de demonios, dependiendo de las circunstancias. Debería ser posible, por ejemplo, ejecutar solo un conjunto mínimo de servicios para realizar tareas de mantenimiento del sistema.

Note

Estrictamente hablando, el sistema operativo es solo el núcleo y sus componentes que controlan los dispositivos y gestionan todos los procesos. Sin embargo, es común usar el término "sistema operativo" de manera más flexible, para designar un grupo completo de programas distintos que componen el entorno de software donde el usuario puede realizar las tareas computacionales básicas.

La inicialización del sistema operativo comienza cuando el gestor de arranque carga el núcleo en la RAM. Luego, el núcleo se hará cargo de la CPU y comenzará a detectar y configurar los aspectos fundamentales del sistema operativo, como la configuración básica de los dispositivos y el direccionamiento de la memoria.

El núcleo del sistema operativo abrirá el *initramfs* (*initial RAM filesystem*). Initramfs es un archivo que contiene un sistema de archivos utilizado como un sistema de archivos raíz temporal durante el proceso de arranque. El objetivo principal de un archivo initramfs es proporcionar los módulos necesarios para que el núcleo pueda acceder al sistema de archivos raíz "real" del sistema operativo.

Tan pronto como el sistema de archivos raíz esté disponible, el núcleo montará todos los sistemas de archivos configurados en `/etc/fstab` y luego ejecutará el primer programa, una utilidad llamada `init`. El programa `init` es responsable de ejecutar todos los scripts de inicialización y demonios del sistema. Existen implementaciones distintas de tales iniciadores de sistemas aparte del init tradicional, como *systemd* y *Upstart*. Una vez que se carga el programa init, initramfs se elimina de la RAM.

SysV standard

Un administrador de servicios basado en el estándar SysVinit controla qué demonios y recursos estarán disponibles empleando el concepto de *runlevels*. Los niveles de ejecución están numerados del 0 al 6 y están diseñados por los encargados de la distribución para cumplir con propósitos específicos. Las únicas definiciones de nivel de ejecución compartidas entre todas las distribuciones son los niveles de ejecución 0, 1 y 6.

systemd

systemd es un administrador moderno de sistemas y servicios con una capa de compatibilidad para los comandos y niveles de ejecución de SysV. systemd tiene una estructura concurrente, emplea sockets y D-Bus para la activación del servicio, ejecución de demonios a demanda, monitoreo de procesos con *cgroups*, *snapshot support*, recuperación de sesión del sistema, control de punto de montaje y un control de servicio basado en la dependencia. En los últimos años, la mayoría de las principales distribuciones de Linux han adoptado gradualmente systemd como su administrador de sistema predeterminado.

Upstart

Al igual que systemd, Upstart es un sustituto de init. El objetivo de Upstart es acelerar el proceso de arranque paralelizando el proceso de carga de los servicios del sistema. Upstart fue utilizado por distribuciones basadas en Ubuntu en versiones anteriores, pero hoy dio paso a systemd.

Inspección de Inicialización

Pueden ocurrir errores durante el proceso de arranque, pero pueden no ser tan críticos para detener completamente el sistema operativo. No obstante, estos errores pueden comprometer el comportamiento esperado del sistema. Todos los errores dan como resultado mensajes que pueden usarse para futuras investigaciones, ya que contienen información valiosa sobre cuándo y cómo ocurrió el error. Incluso cuando no se generan mensajes de error, la información recopilada durante el proceso de arranque puede ser útil para fines de ajuste y configuración.

El espacio de memoria donde el kernel almacena sus mensajes, incluidos los mensajes de arranque, se llama *kernel ring buffer*. Los mensajes se guardan en el búfer del anillo del núcleo incluso cuando no se muestran durante el proceso de inicialización, como cuando se muestra una animación en su lugar. Sin embargo, el buffer del anillo del núcleo pierde todos los mensajes cuando el sistema está apagado o al ejecutar el comando `dmesg --clear`. Sin opciones, el comando `dmesg` muestra los mensajes actuales en el búfer del núcleo:

```
$ dmesg
[    5.262389] EXT4-fs (sda1): mounted filesystem with ordered data
mode. Opts: (null)
[    5.449712] ip_tables: (C) 2000-2006 Netfilter Core Team
[    5.460286] systemd[1]: systemd 237 running in system mode.
[    5.480138] systemd[1]: Detected architecture x86-64.
[    5.481767] systemd[1]: Set hostname to <torre>.
[    5.636607] systemd[1]: Reached target User and Group Name Lookups.
[    5.636866] systemd[1]: Created slice System Slice.
[    5.637000] systemd[1]: Listening on Journal Audit Socket.
[    5.637085] systemd[1]: Listening on Journal Socket.
[    5.637827] systemd[1]: Mounting POSIX Message Queue File System...
[    5.638639] systemd[1]: Started Read required files in advance.
[    5.641661] systemd[1]: Starting Load Kernel Modules...
[    5.661672] EXT4-fs (sda1): re-mounted. Opts: errors=remount-ro
[    5.694322] lp: driver loaded but no devices found
[    5.702609] ppdev: user-space parallel port driver
[    5.705384] parport_pc 00:02: reported by Plug and Play ACPI
[    5.705468] parport0: PC-style at 0x378 (0x778), irq 7, dma 3
[PCSPP,TRISTATE,COMPAT,EPP,ECP,DMA]
[    5.800146] lp0: using parport0 (interrupt-driven).
[    5.897421] systemd-journald[352]: Received request to flush
runtime journal from PID 1
```

La salida de `dmesg` puede tener cientos de líneas, por lo que la lista anterior contiene solo el extracto que muestra el núcleo que llama al administrador del servicio systemd. Los valores al comienzo de las líneas son la cantidad de segundos relativos al inicio de la carga del núcleo.

En sistemas basados en systemd, el comando `journalctl` mostrará los mensajes de inicialización con las opciones `-b`, `--boot`,`-k` o `--dmesg`. El comando `journalctl --`

`list-boots` muestra una lista de números de arranque relativos al arranque actual, su hash de identificación y las marcas de tiempo del primer y último mensaje correspondientes:

```
$ journalctl --list-boots
 -4 9e5b3eb4952845208b841ad4dbefa1a6 Thu 2019-10-03 13:39:23 -03—Thu
2019-10-03 13:40:30 -03
 -3 9e3d79955535430aa43baa17758f40fa Thu 2019-10-03 13:41:15 -03—Thu
2019-10-03 14:56:19 -03
 -2 17672d8851694e6c9bb102df7355452c Thu 2019-10-03 14:56:57 -03—Thu
2019-10-03 19:27:16 -03
 -1 55c0d9439bfb4e85a20a62776d0dbb4d Thu 2019-10-03 19:27:53 -03—Fri
2019-10-04 00:28:47 -03
  0 08fbbebd9f964a74b8a02bb27b200622 Fri 2019-10-04 00:31:01 -03—Fri
2019-10-04 10:17:01 -03
```

Los registros de inicialización anteriores también se mantienen en sistemas basados en systemd, por lo que los mensajes de sesiones anteriores del sistema operativo aún se pueden inspeccionar. Si se proporcionan las opciones `-b 0` o `--boot=0`, se mostrarán los mensajes para el arranque actual. Las opciones `-b -1` o `--boot=-1` mostrarán mensajes de la inicialización anterior. Las opciones `-b -2` o `--boot=-2` mostrarán los mensajes de la inicialización antes de eso y así sucesivamente. El siguiente extracto muestra el llamado del sistema operativo al administrador del servicio systemd para el último proceso de inicialización:

```
$ journalctl -b 0
oct 04 00:31:01 ubuntu-host kernel: EXT4-fs (sda1): mounted filesystem
with ordered data mode. Opts: (null)
oct 04 00:31:01 ubuntu-host kernel: ip_tables: (C) 2000-2006 Netfilter
Core Team
oct 04 00:31:01 ubuntu-host systemd[1]: systemd 237 running in system
mode.
oct 04 00:31:01 ubuntu-host systemd[1]: Detected architecture x86-64.
oct 04 00:31:01 ubuntu-host systemd[1]: Set hostname to <torre>.
oct 04 00:31:01 ubuntu-host systemd[1]: Reached target User and Group
Name Lookups.
oct 04 00:31:01 ubuntu-host systemd[1]: Created slice System Slice.
oct 04 00:31:01 ubuntu-host systemd[1]: Listening on Journal Audit
Socket.
oct 04 00:31:01 ubuntu-host systemd[1]: Listening on Journal Socket.
oct 04 00:31:01 ubuntu-host systemd[1]: Mounting POSIX Message Queue
File System...
oct 04 00:31:01 ubuntu-host systemd[1]: Started Read required files in
advance.
oct 04 00:31:01 ubuntu-host systemd[1]: Starting Load Kernel
Modules...
oct 04 00:31:01 ubuntu-host kernel: EXT4-fs (sda1): re-mounted. Opts:
commit=300,barrier=0,errors=remount-ro
oct 04 00:31:01 ubuntu-host kernel: lp: driver loaded but no devices
found
oct 04 00:31:01 ubuntu-host kernel: ppdev: user-space parallel port
driver
oct 04 00:31:01 ubuntu-host kernel: parport_pc 00:02: reported by Plug
and Play ACPI
oct 04 00:31:01 ubuntu-host kernel: parport0: PC-style at 0x378
(0x778), irq 7, dma 3 [PCSPP,TRISTATE,COMPAT,EPP,ECP,DMA]
oct 04 00:31:01 ubuntu-host kernel: lp0: using parport0 (interrupt-
driven).
```

```
oct 04 00:31:01 ubuntu-host systemd-journald[352]: Journal started
oct 04 00:31:01 ubuntu-host systemd-journald[352]: Runtime journal
(/run/log/journal/abb765408f3741ae9519ab3b96063a15) is 4.9M, max
39.4M, 34.5M free.
oct 04 00:31:01 ubuntu-host systemd-modules-load[335]: Inserted module
'lp'
oct 04 00:31:01 ubuntu-host systemd-modules-load[335]: Inserted module
'ppdev'
oct 04 00:31:01 ubuntu-host systemd-modules-load[335]: Inserted module
'parport_pc'
oct 04 00:31:01 ubuntu-host systemd[1]: Starting Flush Journal to
Persistent Storage...
```

La inicialización y otros mensajes emitidos por el sistema operativo se almacenan en archivos dentro del directorio `/var/log/`. Si ocurre un error crítico y el sistema operativo no puede continuar el proceso de inicialización después de cargar el kernel y el initramfs, se podría usar un medio de arranque alternativo para iniciar el sistema y acceder al sistema de archivos correspondiente. Luego, los archivos almacendos en `/var/log/` pueden buscarse por posibles razones que causan la interrupción del proceso de arranque. Las opciones `-D` o `--directory` del comando `journalctl` se pueden usar para leer mensajes de registro en directorios que no sean `/var/log/journal/`, que es la ubicación predeterminada para los mensajes de registro de systemd. Como los mensajes de registro de systemd se almacenan en texto sin formato, se requiere el comando `journalctl` para leerlos.

Ejercicios Guiados

1. En una máquina equipada con un firmware de BIOS, ¿dónde se encuentra el binario de arranque?

2. El firmware UEFI admite funciones ampliadas proporcionadas por programas externos, llamadas aplicaciones EFI. Estas aplicaciones, sin embargo, tienen su propia ubicación especial. ¿En qué parte del sistema se ubicarían las aplicaciones EFI?

3. Los cargadores de arranque permiten pasar parámetros personalizados del kernel antes de cargarlo. Suponga que el sistema no puede iniciarse debido a una ubicación errónea del sistema de archivos raíz. ¿Cómo se le daría al kernel el sistema de archivos raíz correcto, ubicado en `/dev/sda3`?

4. El proceso de arranque de una máquina Linux termina con el siguiente mensaje:

   ```
   ALERT! /dev/sda3 does not exist. Dropping to a shell!
   ```

   ¿Cuál es la causa probable de este problema?

Ejercicios Exploratorios

1. El gestor de arranque presentará una lista de sistemas operativos para elegir cuando se instala más de un sistema operativo en la máquina. Sin embargo, un sistema operativo recién instalado puede sobrescribir el MBR del disco duro, borrando la primera etapa del gestor de arranque y haciendo que el otro sistema operativo sea inaccesible. ¿Por qué esto no sucedería en una máquina equipada con un firmware UEFI?

2. ¿Cuál es una consecuencia común de instalar un kernel personalizado sin proporcionar una imagen initramfs adecuada?

3. El registro de inicialización tiene cientos de líneas, por lo que la salida del comando `dmesg` a menudo se canaliza a un método de lectura más ágil, como el comando `less`. ¿Qué opción `dmesg` paginará automáticamente su salida, eliminando la necesidad de usar un comando de paginación explícitamente?

4. Un disco duro que contiene todo el sistema de archivos de una máquina desconectado se removió y se conectó a una máquina en funcionamiento como unidad secundaria. Suponiendo que su punto de montaje es `/mnt/hd`, ¿cómo se usaría `journalctl` para inspeccionar el contenido de los archivos de logs ubicados en `/mnt/hd/var/log/journal/`?

Resumen

Esta lección cubre la secuencia de arranque en un sistema Linux estándar. El conocimiento adecuado de cómo funciona el proceso de arranque de un sistema Linux ayuda a evitar errores que pueden hacer que el sistema sea inaccesible. La lección trata los siguientes temas:

- ¿Cómo difieren los métodos de arranque BIOS y UEFI?
- Etapas típicas de inicialización del sistema.
- Mensajes de recuperación de arranque.

Los comandos y procedimientos abordados fueron:

- Parámetros comunes del núcleo del sistema operativo.
- Comandos para leer mensajes de arranque: `dmesg` y `journalctl`.

Respuestas a los ejercicios guiados

1. En una máquina equipada con un firmware de BIOS, ¿dónde se encuentra el binario de arranque?

   En el MBR del primer dispositivo de almacenamiento, como se define en la utilidad de configuración del BIOS.

2. El firmware UEFI admite funciones ampliadas proporcionadas por programas externos, llamadas aplicaciones EFI. Estas aplicaciones, sin embargo, tienen su propia ubicación especial. ¿En qué parte del sistema se ubicarían las aplicaciones EFI?

   Las aplicaciones EFI se almacenan en la partición del sistema EFI (ESP), ubicada en cualquier bloque de almacenamiento disponible con un sistema de archivos compatible (generalmente un sistema de archivos FAT32).

3. Los cargadores de arranque permiten pasar parámetros personalizados del kernel antes de cargarlo. Suponga que el sistema no puede iniciarse debido a una ubicación errónea del sistema de archivos raíz. ¿Cómo se le daría al kernel el sistema de archivos raíz correcto, ubicado en `/dev/sda3`?

   Se debe usar el parámetro `root`, como `root=/dev/sda3`.

4. El proceso de arranque de una máquina Linux termina con el siguiente mensaje:

   ```
   ALERT! /dev/sda3 does not exist. Dropping to a shell!
   ```

   ¿Cuál es la causa probable de este problema?

   El núcleo no pudo encontrar el dispositivo `/dev/sda3`, determinado como el sistema de archivos raíz.

Respuestas a ejercicios exploratorios

1. El gestor de arranque presentará una lista de sistemas operativos para elegir cuando se instala más de un sistema operativo en la máquina. Sin embargo, un sistema operativo recién instalado puede sobrescribir el MBR del disco duro, borrando la primera etapa del gestor de arranque y haciendo que el otro sistema operativo sea inaccesible. ¿Por qué esto no sucedería en una máquina equipada con un firmware UEFI?

   Las máquinas UEFI no usan el MBR del disco duro para almacenar la primera etapa del gestor de arranque.

2. ¿Cuál es una consecuencia común de instalar un kernel personalizado sin proporcionar una imagen initramfs adecuada?

El sistema de archivos raíz puede ser inaccesible si fue compilado como un módulo de kernel externo.

3. El registro de inicialización tiene cientos de líneas, por lo que la salida del comando `dmesg` a menudo se canaliza a un método de lectura más ágil, como el comando `less`. ¿Qué opción `dmesg` paginará automáticamente su salida, eliminando la necesidad de usar un comando de paginación explícitamente?

   Los comandos `dmesg -H` o `dmesg --human` habilitarán el paginador por defecto.

4. Un disco duro que contiene todo el sistema de archivos de una máquina desconectada se removió y se conectó a una máquina en funcionamiento como unidad secundaria. Suponiendo que su punto de montaje es `/mnt/hd`, ¿cómo se usaría `journalctl` para inspeccionar el contenido de los archivos de diario ubicados en `/mnt/hd/var/log/journal/`?

   Con los comandos `journalctl -D /mnt/hd/var/log/journal` o `journalctl --directory=/mnt/hd/var/log/journal`

101.3 Cambiar los niveles de ejecución / objetivos de arranque y apagar o reiniciar el sistema

Referencia al objetivo del LPI

LPIC-1 v5, Exam 101, Objective 101.3

Importancia

3

Áreas de conocimiento clave

- Establecer el nivel de ejecución o el objetivo de arranque predeterminado.
- * Cambiar entre niveles de ejecución / objetivos de arranque, incluido el modo monousuario. Apagar y reiniciar desde la línea de comandos.
- Alertar a los usuarios antes de cambiar de nivel de ejecución/objetivo de arranque u otros eventos importantes del sistema.
- Terminar procesos de forma adecuada.
- Conocimiento de acpid.

Lista parcial de archivos, términos y utilidades

- `/etc/inittab`
- `shutdown`
- `init`
- `/etc/init.d/`
- `telinit`
- `systemd`
- `systemctl`
- `/etc/systemd/`
- `/usr/lib/systemd/`
- `wall`

101.3 Lección 1

Certificación:	LPIC-1
Versión:	5.0
Tema:	101 Arquitectura de Sistema
Objetivo:	101.3 Cambiar los niveles de ejecución / objetivos de arranque y apagar o reiniciar el sistema
Lección:	1 de 1

Introducción

Una característica común entre los sistemas operativos que siguen los principios de diseño de Unix es el empleo de procesos separados para controlar distintas funciones del sistema. Estos procesos, llamados *daemons* (o, más generalmente, *services*), también son responsables de las características extendidas subyacentes al sistema operativo, como los servicios de aplicaciones de red (servidor HTTP, intercambio de archivos, correo electrónico, etc.), bases de datos, configuración bajo demanda, etc. Aunque Linux utiliza un núcleo monolítico, muchos aspectos de bajo nivel del sistema operativo se ven afectados por demonios, como el equilibrio de carga y la configuración del firewall.

Los demonios (*daemons*) que deberían estar activos dependen del propósito del sistema. El conjunto de demonios activos también debe poder modificarse en tiempo de ejecución, de modo que los servicios se puedan iniciar y detener sin tener que reiniciar todo el sistema. Para abordar este problema, cada distribución principal de Linux ofrece alguna forma o utilidad de administración de servicios para controlar el sistema.

Los servicios pueden ser controlados por scripts de shell o por un programa y sus archivos de configuración compatibles. El primer método lo implementa el estándar *SysVinit*, también conocido como *System V* o simplemente *SysV*. El segundo método es implementado por *systemd* y *Upstart*. Históricamente, los administradores de servicios basados en SysV fueron los más utilizados por las distribuciones de Linux. Hoy en día, los administradores de servicios basados en sistemas se encuentran con mayor frecuencia en la mayoría de las distribuciones de Linux. El administrador de servicios es el primer programa lanzado por el núcleo durante el proceso de arranque, por lo que su PID (número de identificación del proceso) siempre es `1`.

Estándar SysVinit

Un administrador de servicios basado en el estándar SysVinit proporcionará conjuntos predefinidos de estados del sistema, llamados *runlevels*, y sus correspondientes archivos de script de servicio para ser ejecutados. Los niveles de ejecución están numerados de `0` a `6`, y generalmente se asignan a los siguientes propósitos:

Runlevel 0

Apagado del sistema.

Runlevel 1, s o usuario único

Modo de usuario único, sin red y otras capacidades no esenciales (modo de mantenimiento).

Runlevel 2, 3 o 4

Modo multiusuario. Los usuarios pueden iniciar sesión por consola o red. Los niveles de ejecución 2 y 4 no se usan con frecuencia.

Runlevel 5

Modo multiusuario. Es equivalente a 3, más el inicio de sesión en modo gráfico.

Runlevel 6

Reinicio del sistema.

El programa responsable de administrar los niveles de ejecución y los demonios/recursos asociados es `/sbin/init`. Durante la inicialización del sistema, el programa `init` identifica el nivel de ejecución solicitado, definido por un parámetro del núcleo del sistema operativo o en el archivo `/etc/inittab`, y carga los scripts asociados que se enumeran allí para el nivel de ejecución dado. Cada nivel de ejecución puede tener muchos archivos de servicio asociados, generalmente scripts en el directorio `/etc/init.d/`. Como no todos los niveles de ejecución son equivalentes a través de diferentes distribuciones de Linux, también se puede encontrar una breve descripción del propósito del nivel de ejecución en las distribuciones basadas en SysV.

La sintaxis del archivo `/etc/inittab` usa este formato:

```
id:runlevels:action:process
```

El `id` es un nombre genérico de hasta cuatro caracteres de longitud utilizado para identificar la entrada. La entrada `runlevels` es una lista de números de niveles para los que se debe ejecutar una acción específica. El término `action` define cómo `init` ejecutará el proceso indicado por el término `process`. Las acciones disponibles son:

`boot`

El proceso se ejecutará durante la inicialización del sistema. El campo `runlevels` se ignora.

`bootwait`

El proceso se ejecutará durante la inicialización del sistema e `init` esperará hasta que termine para continuar. El campo `runlevels` se ignora.

`sysinit`

El proceso se ejecutará después de la inicialización del sistema, independientemente del nivel de ejecución. El campo `runlevels` se ignora.

`wait`

El proceso se ejecutará para los niveles de ejecución dados e `init` esperará hasta que termine para continuar.

`respawn`

El proceso se reiniciará si finaliza.

`ctrlaltdel`

El proceso se ejecutará cuando el proceso init reciba la señal `SIGINT`, que se activará cuando se presione la secuencia de teclas `Ctrl`+`Alt`+`Supr`.

El nivel de ejecución predeterminado, el que se elegirá si no se proporciona otro como parámetro del núcleo, también se define en `/etc/inittab`, en la entrada `id:x:initdefault`. La `x` es el número del nivel de ejecución predeterminado. Este número nunca debe ser `0` o `6`, ya que provocaría que el sistema se apague o reinicie tan pronto como finalice el proceso de arranque. A continuación se muestra un archivo típico `/etc/inittab`:

```
# Nivel de ejecución predeterminado
id:3:initdefault:

# Script de configuración ejecutado durante el arranque
si::sysinit:/etc/init.d/rcS

# Acción tomada en el nivel de ejecución S (usuario único)
~:S:wait:/sbin/sulogin

# Configuración para cada nivel de ejecución
l0:0:wait:/etc/init.d/rc 0
l1:1:wait:/etc/init.d/rc 1
l2:2:wait:/etc/init.d/rc 2
l3:3:wait:/etc/init.d/rc 3
l4:4:wait:/etc/init.d/rc 4
l5:5:wait:/etc/init.d/rc 5
l6:6:wait:/etc/init.d/rc 6

# Acción tomada sobre teclado ctrl+alt+del
ca::ctrlaltdel:/sbin/shutdown -r now

# Habilitar consolas para los niveles de ejecución 2 y 3
1:23:respawn:/sbin/getty tty1 VC linux
2:23:respawn:/sbin/getty tty2 VC linux
3:23:respawn:/sbin/getty tty3 VC linux
4:23:respawn:/sbin/getty tty4 VC linux

# Para el nivel de ejecución 3, también habilite serial
# terminales ttyS0 y ttyS1 (módem) consolas
S0:3:respawn:/sbin/getty -L 9600 ttyS0 vt320
S1:3:respawn:/sbin/mgetty -x0 -D ttyS1
```

El comando `telinit q` debe ejecutarse cada vez que se modifica el archivo `/etc/inittab`. El argumento `q` (o `Q`) le dice a init que vuelva a cargar su configuración. Este paso es importante para evitar que el sistema se detenga debido a una configuración incorrecta en `/etc/inittab`.

Los scripts utilizados por `init` para configurar cada nivel de ejecución se almacenan en el directorio `/etc/init.d/`. Cada nivel de ejecución tiene un directorio asociado en `/etc/`, llamado `/etc/rc0.d/`, `/etc/rc1.d/`, `/etc/rc2.d/`, etc., con los scripts eso debería ejecutarse cuando comience el nivel de ejecución correspondiente. Como el mismo script puede ser usado por diferentes niveles de ejecución, los archivos en esos directorios son solo enlaces simbólicos a los scripts reales en `/etc/init.d/`. Además, la primera letra del nombre de archivo del enlace en el directorio del nivel de ejecución indica si el servicio debe iniciarse o terminarse para el nivel de ejecución correspondiente. El nombre de archivo de un enlace que comienza con la letra `K` determina que el servicio se eliminará al ingresar el nivel de ejecución (kill). Comenzando con la letra `S`, el servicio se iniciará al ingresar el nivel de ejecución (inicio). El directorio `/etc/rc1.d/`, por ejemplo, tendrá muchos enlaces a los scripts de red que comienzan con la letra `K`, considerando que el nivel de ejecución 1 corresponde a un solo usuario, sin conectividad de red.

El comando `runlevel` muestra el nivel de ejecución actual para el sistema. El comando `runlevel` muestra dos valores, el primero es el nivel de ejecución anterior y el segundo es el nivel de ejecución actual:

```
$ runlevel
N 3
```

La letra `N` en la salida muestra que el nivel de ejecución no ha cambiado desde el último arranque. En el ejemplo, el `runlevel 3` es el nivel de ejecución actual del sistema.

El mismo programa `init` puede usarse para alternar entre niveles de ejecución en un sistema en ejecución, sin la necesidad de reiniciar. El comando `telinit` también se puede usar para alternar entre los niveles de ejecución. Por ejemplo, los comandos `telinit 1`,`telinit s` o `telinit S` cambiarán el sistema al nivel de ejecución 1.

systemd

Actualmente, systemd es el conjunto de herramientas más utilizado para administrar los recursos y servicios del sistema, que systemd denomina unidades (*units*). Una unidad consta de nombre, tipo y un archivo de configuración correspondiente. Por ejemplo, la unidad para un proceso de servidor *httpd* (como el servidor web Apache) será `httpd.service` en distribuciones basadas en Red Hat, su archivo de configuración también se llamará `httpd.service` (en distribuciones basadas en Debian esta unidad es llamada `apache2.service`).

Hay siete tipos distintos de unidades systemd:

`service`

El tipo de unidad más común, para recursos activos del sistema que se pueden iniciar, interrumpir y recargar.

`socket`

El tipo de unidad de socket puede ser un socket de sistema de archivos o un socket de red. Todas las unidades de socket tienen una unidad de servicio correspondiente, cargada cuando el socket recibe una solicitud.

`device`

Una unidad de dispositivo está asociada con un dispositivo de hardware identificado por el núcleo. Un dispositivo solo se tomará como una unidad systemd si existe una regla udev para este propósito. Se puede usar una unidad de dispositivo para resolver dependencias de configuración cuando se detecta cierto hardware, dado que las propiedades de la regla udev se pueden usar como parámetros para la unidad de dispositivo.

`mount`

Una unidad de montaje es una definición de punto de montaje en el sistema de archivos, similar a una entrada en `/etc/fstab`.

`automount`

Una unidad de montaje automático también es una definición de punto de montaje en el sistema de archivos, pero se monta automáticamente. Cada unidad de montaje automático tiene una unidad de montaje correspondiente, que se inicia cuando se accede al punto de montaje automático.

`target`

Una unidad target es una agrupación de otras unidades, administradas como una sola unidad.

`snapshot`

Una unidad snapshot es un estado guardado del administrador del sistema (no disponible en todas las distribuciones de Linux).

El comando principal para controlar las unidades systemd es `systemctl`. El comando `systemctl` se usa para ejecutar todas las tareas relacionadas con la activación, desactivación, ejecución, interrupción, monitoreo de la unidad, etc. Para una unidad ficticia llamada `unit.service`, por ejemplo, las acciones más comunes de `systemctl` serán:

`systemctl start unit.service`

Inicia una unidad (`unit`).

`systemctl stop unit.service`

Detiene una unidad (`unit`).

`systemctl restart unit.service`

Reinicia una unidad (`unit`).

`systemctl status unit.service`

Muestra el estado de la unidad (`unit`), incluyendo si está en ejecución o no.

`systemctl is-active unit.service`

Muestra *active* Si la unidad (`unit`) está en ejecución o inactiva.

`systemctl enable unit.service`

La unidad (`unit`) se cargará durante la inicialización del sistema.

`systemctl disable unit.service`

La unidad (`unit`) no se cargará durante la inicialización del sistema.

`systemctl is-enabled unit.service`

Verifica si la unidad (`unit`) comienza con el sistema. La respuesta se almacena en la variable `$?`. El valor `0` indica que `unit` comienza con el sistema y el valor `1` indica que `unit` no comienza con el sistema.

Note

Las instalaciones más recientes de systemd en realidad enumerarán la configuración de una unidad para el tiempo de arranque. Por ejemplo:

```
$ systemctl is-enabled apparmor.service
enabled
```

Si no existen otras unidades con el mismo nombre en el sistema, el sufijo después del punto se puede quitar. Si, por ejemplo, solo hay una unidad `httpd` de tipo `service`, entonces solo `httpd` es suficiente como parámetro de unidad para `systemctl`.

El comando `systemctl` también puede controlar *system targets*. La unidad `multi-user.target`, por ejemplo, combina todas las unidades requeridas por el entorno del sistema multiusuario. Es similar al nivel de ejecución número 3 en un sistema que utiliza SysV.

El comando `systemctl isolate` alterna entre diferentes objetivos. Por lo tanto, para alternar manualmente al objetivo `multiusuario`:

```
# systemctl isolate multi-user.target
```

Hay objetivos correspondientes a los niveles de ejecución de SysV, comenzando con `runlevel0.target` hasta `runlevel6.target`. Sin embargo, systemd no utiliza el

archivo `/etc/inittab`. Para cambiar el objetivo predeterminado del sistema, la opción `systemd.unit` se puede agregar a la lista de parámetros del núcleo del sistema operativo. Por ejemplo, para usar `multi-user.target` como objetivo estándar, el parámetro del núcleo del sistema operativo debe ser `systemd.unit=multi-user.target`. Todos los parámetros del kernel pueden hacerse persistentes cambiando la configuración del gestor de arranque.

Otra forma de cambiar el objetivo predeterminado es modificar el enlace simbólico `/etc/systemd/system/default.target` para que apunte al objetivo deseado. La redefinición del enlace se puede hacer con el comando `systemctl` por sí mismo:

```
# systemctl set-default multi-user.target
```

Del mismo modo, puede determinar cuál es el objetivo de arranque predeterminado de su sistema con el siguiente comando:

```
$ systemctl get-default
graphical.target
```

Similar a los sistemas que adoptan SysV, el objetivo predeterminado nunca debe apuntar a `shutdown.target`, ya que corresponde al nivel de ejecución 0 (apagado).

Los archivos de configuración asociados con cada unidad se pueden encontrar en el directorio `/lib/systemd/system/`. El comando `systemctl list-unit-files` enumera todas las unidades disponibles y muestra si están habilitadas para iniciarse cuando se inicia el sistema. La opción `--type` seleccionará solo las unidades para un tipo dado, como en `systemctl list-unit-files --type=service` y `systemctl list-unit-files --type=target`.

Las unidades activas o unidades que han estado activas durante la sesión actual del sistema se pueden enumerar con el comando `systemctl list-units`. Al igual que la opción `list-unit-files`, el comando `systemctl list-units --type=service` seleccionará solo unidades de tipo `service` y el comando `systemctl list-units --type=target` seleccionará solo unidades de tipo `target`.

systemd también es responsable de desencadenar y responder a eventos relacionados con la energía del sistema. El comando `systemctl suspend` pondrá el sistema en modo de bajo consumo, manteniendo los datos actuales en la memoria. El comando `systemctl hibernate` copiará todos los datos de la memoria al disco, por lo que el estado actual del sistema se puede recuperar después de apagarlo. Las acciones asociadas con tales eventos se definen en el archivo `/etc/systemd/logind.conf` o en archivos separados dentro del directorio `/etc/systemd/logind.conf.d/`. Sin embargo, esta función systemd solo se puede usar cuando no hay otro administrador de energía ejecutándose en el sistema, como el demonio `acpid`. El demonio `acpid` es el principal administrador de energía para Linux y permite ajustes más precisos a las acciones posteriores a eventos relacionados con la energía, como cerrar la tapa del portátil, batería baja o niveles de carga de la batería.

Upstart

Los scripts de inicialización utilizados por Upstart se encuentran en el directorio `/etc/init/`. Los servicios del sistema se pueden enumerar con el comando `initctl list`, que también muestra el estado actual de los servicios y, si está disponible, su número PID.

```
# initctl list
avahi-cups-reload stop/waiting
avahi-daemon start/running, process 1123
mountall-net stop/waiting
mountnfs-bootclean.sh start/running
nmbd start/running, process 3085
passwd stop/waiting
rc stop/waiting
rsyslog start/running, process 1095
tty4 start/running, process 1761
udev start/running, process 1073
upstart-udev-bridge start/running, process 1066
console-setup stop/waiting
irqbalance start/running, process 1842
plymouth-log stop/waiting
smbd start/running, process 1457
tty5 start/running, process 1764
failsafe stop/waiting
```

Cada acción de Upstart tiene su propio comando independiente. Por ejemplo, el comando `start` puede usarse para iniciar la sexta terminal virtual:

```
# start tty6
```

El estado actual de un recurso se puede verificar con el comando `status`:

```
# status tty6
tty6 start/running, process 3282
```

Y la interrupción de un servicio se realiza con el comando `stop`:

```
# stop tty6
```

Upstart no usa el archivo `/etc/inittab` para definir los niveles de ejecución, pero los comandos heredados `runlevel` y `telinit` todavía pueden usarse para verificar y alternar entre los niveles de ejecución.

Note: Upstart fue desarrollado para la distribución Ubuntu Linux para ayudar a facilitar el inicio paralelo de los procesos. Ubuntu ha dejado de usar Upstart desde 2015 cuando cambió de Upstart a systemd.

Apagado y Reinicio

Un comando muy tradicional utilizado para apagar o reiniciar el sistema se llama `shutdown`. El comando `shutdown` agrega funciones adicionales al proceso de apagado: notifica automáticamente a todos los usuarios conectados con un mensaje de advertencia en sus sesiones de shell y se evitan nuevos inicios de sesión. El comando

`shutdown` actúa como intermediario para los procedimientos SysV o systemd, es decir, ejecuta la acción solicitada llamando a la acción correspondiente en el administrador de servicios adoptado por el sistema.

Después de ejecutar `shutdown`, todos los procesos reciben la señal `SIGTERM`, seguida de la señal `SIGKILL`, luego el sistema se apaga o cambia su nivel de ejecución. Por defecto, cuando no se utilizan las opciones `-h` o `-r`, el sistema alterna al nivel de ejecución 1, es decir, el modo de usuario único. Para cambiar las opciones predeterminadas para `shutdown`, el comando debe ejecutarse con la siguiente sintaxis:

```
$ shutdown [option] time [message]
```

Solo se requiere el parámetro `time`. El parámetro `time` define cuándo se ejecutará la acción solicitada, aceptando los siguientes formatos:

`hh:mm`

Este formato especifica el tiempo de ejecución como hora y minutos.

`+m`

Este formato especifica cuántos minutos esperar antes de la ejecución.

`now` o `+0`

Este formato determina la ejecución inmediata.

El parámetro `message` es el texto de advertencia enviado a todas las sesiones de terminal de los usuarios conectados.

La implementación de SysV permite limitar a los usuarios que podrán reiniciar la máquina presionando `Ctrl+Alt+Supr`. Esto es posible colocando la opción `-a` para el comando `shutdown` presente en la línea con respecto a `ctrlaltdel` en el archivo `/etc/inittab`. Al hacer esto, solo los usuarios cuyos nombres de usuario estén en el archivo `/etc/shutdown.allow` podrán reiniciar el sistema con la combinación de teclas `Ctrl+Alt+Supr`.

El comando `systemctl` también se puede usar para apagar o reiniciar la máquina en sistemas que emplean systemd. Para reiniciar el sistema, se debe usar el comando `systemctl reboot`. Para apagar el sistema, se debe usar el comando `systemctl poweroff`. Ambos comandos requieren privilegios de root para ejecutarse, ya que los usuarios comunes no pueden realizar dichos procedimientos.

Note

Algunas distribuciones de Linux vincularán `poweroff` y `reboot` a `systemctl` como comandos individuales. Por ejemplo:

```
$ sudo which poweroff
/usr/sbin/poweroff
$ sudo ls -l /usr/sbin/poweroff
```

```
lrwxrwxrwx 1 root root 14 Aug 20 07:50 /usr/sbin/poweroff ->
/bin/systemctl
```

No todas las actividades de mantenimiento requieren que el sistema se apague o reinicie. Sin embargo, cuando es necesario cambiar el estado del sistema al modo de usuario único, es importante advertir a los usuarios que han iniciado sesión para que no se vean perjudicados por la finalización brusca de sus actividades.

De manera similar a lo que hace el comando `shutdown` cuando se apaga o reinicia el sistema, el comando `wall` puede enviar un mensaje a las sesiones de terminal de todos los usuarios conectados. Para hacerlo, el administrador del sistema solo necesita proporcionar un archivo o escribir directamente el mensaje como parámetro para ordenar `wall`.

Ejercicios Guiados

1. ¿Cómo podría usarse el comando `telinit` para reiniciar el sistema?

2. ¿Qué pasará con los servicios relacionados con el archivo `/etc/rc1.d/K90network` cuando el sistema ingrese al nivel de ejecución 1?

3. Usando el comando `systemctl`, ¿cómo podría un usuario verificar si la unidad `sshd.service` está funcionando?

4. En un sistema basado en systemd, ¿qué comando debe ejecutarse para permitir la activación de la unidad `sshd.service` durante la inicialización del sistema?

Ejercicios Exploratorios

1. En un sistema basado en SysV, suponga que el nivel de ejecución predeterminado definido en `/etc/inittab` es 3, pero el sistema siempre comienza en el nivel de ejecución 1. ¿Cuál es la causa probable de eso?

2. Aunque el archivo `/sbin/init` se puede encontrar en sistemas basados en systemd, es solo un enlace simbólico a otro archivo ejecutable. En tales sistemas, ¿cuál es el archivo señalado por `/sbin/init`?

3. ¿Cómo se puede verificar el objetivo predeterminado del sistema (system_target) en un sistema basado en systemd?

4. ¿Cómo se puede cancelar un reinicio del sistema programado con el comando `shutdown`?

Resumen

Esta lección cubre las principales utilidades utilizadas como administradores de servicios por las distribuciones de Linux. Las utilidades SysVinit, systemd y Upstart tienen su propio enfoque para controlar los servicios del sistema y los estados del sistema. La lección trata los siguientes temas:

- ¿Qué son los servicios del sistema y su papel en el sistema operativo?
- Conceptos y uso básico de los comandos SysVinit, systemd y Upstart.
- ¿Cómo iniciar, detener y reiniciar correctamente los servicios del sistema y el sistema mismo?

Los comandos y procedimientos abordados fueron:

- Comandos y archivos relacionados con SysVinit, como `init`, `/etc/inittab` y `telinit`.
- El comando principal de systemd: `systemctl`.
- Comandos de inicio: `initctl`, `status`, `start`, `stop`.
- Comandos tradicionales de administración de energía, como `shutdown` y `wall`.

Respuestas a los ejercicios guiados

1. ¿Cómo podría usarse el comando `telinit` para reiniciar el sistema?

   El comando `telinit 6` se alternará con el nivel de ejecución 6, es decir, reiniciar el sistema.

2. ¿Qué pasará con los servicios relacionados con el archivo `/etc/rc1.d/K90network` cuando el sistema ingrese al nivel de ejecución 1?

   Debido a la letra `K` al comienzo del nombre del archivo, los servicios relacionados se detendrán.

3. Usando el comando `systemctl`, ¿cómo podría un usuario verificar si la unidad `sshd.service` está funcionando?

   Con el comando `systemctl status sshd.service` o `systemctl is-active sshd.service`.

4. En un sistema basado en systemd, ¿qué comando debe ejecutarse para permitir la activación de la unidad `sshd.service` durante la inicialización del sistema?

   Comando `systemctl enable sshd.service`, ejecutado por root.

Respuestas a ejercicios exploratorios

1. En un sistema basado en SysV, suponga que el nivel de ejecución predeterminado definido en `/etc/inittab` es 3, pero el sistema siempre comienza en el nivel de ejecución 1. ¿Cuál es la causa probable de eso?

   Los parámetros `1` o `S` pueden estar presentes en la lista de parámetros del núcleo del sistema operativo.

2. Aunque el archivo `/sbin/init` se puede encontrar en sistemas basados en systemd, es solo un enlace simbólico a otro archivo ejecutable. En tales sistemas, ¿cuál es el archivo señalado por `/sbin/init`?

   El binario principal de systemd: `/lib/systemd/systemd`.

3. ¿Cómo se puede verificar el objetivo predeterminado del sistema (system_target) en un sistema basado en systemd?

   El enlace simbólico `/etc/systemd/system/default.target` apuntará al archivo de unidad definido como el objetivo predeterminado. También se puede usar el comando `systemctl get-default`.

4. ¿Cómo se puede cancelar un reinicio del sistema programado con el comando `shutdown`?

   Se debe usar el comando `shutdown -c`.